

Operating System

LAB # 11

Objective

“Execute the Linux Shell scripts”

Shell Scripting

Writing a series of commands in a text file to automate tasks on a Linux system.

/bin/bash

The location of the Bash shell program on Linux systems.

Bash

A popular command-line interpreter in Linux used to interact with the system and write scripts.

E.g You use Bash when you open a terminal and type commands like `cd`, `ls`, or `echo`.

Shell

To automate and simplify repetitive tasks.

E.g Instead of typing commands one by one every day, put them in a script and run the script.

PowerShell in Windows

PowerShell is a cross-platform command-line shell and scripting language developed by Microsoft. It is available for Windows, Linux, and macOS, enabling users to automate and manage their systems consistently across different platforms.

Step 1: Creating a script file

vi first.sh or vi first

Or

cat > first.sh or cat > first

Step 2: Specify the shell path

#!/bin/bash

Step 3: Write the script

echo "Knowledge is Power"

Step 4: Make the script file executable

chmod 755 first.sh / first

Step 6: Run the script

./first.sh **Or** ./first

For comments

Use #

E.g

My first script

Variables in linux

1.System variables

Created and maintained by Linux itself. This type of variable defined in CAPITAL LETTERS.

E.g 1. echo \$HOME 2. echo \$USERNAME

2. User defined variables (UDV)

Created and maintained by the user. This type of variable is defined in lower LETTERS.

E.g 1. age=18 2. echo \$age

Null variables

Null variables can be defined in scripting

E.g 1.age= 2.name=""

Shell Arithmetic

expr 1 + 3

expr 2 - 1

expr 10 / 2

expr 20 % 3 (remainder read as 20 mod 3 and remainder is 2)

expr 10 * 3 (Multiplication use * not * since its wild card)

E.g

echo `expr 6 + 3`

More about Quotes

Quotes	Name	Meaning
“	Double quotes	"Double Quotes" - Anything enclosed in double quotes removes meaning of that characters (except \ and \$).
‘	Single quotes	'Single quotes' - Enclosed in single quotes remains unchanged.
`	Back quote	`Back quote` - To execute a command.

E.g

date is a command which is hold current date

```
echo "Today is date"
```

Output

Today is date

```
echo "Today is `date`"
```

Today is [Current date]

Note : `` is use for built-in command

The “read” Statement

```
echo "Enter Your First Name:" Or echo -n "Enter Your First Name:"
```

```
read firstName
```

```
echo "Hello $firstName"
```

Note : \$ is use for custom variables

Operating System

LAB # 12

Objective

“Execute the Shell script by using Control Structures and Loops”

Comparisons:

In Bash	In actual	Meaning
-eq	==	equal equal to
-ne	!=	not equal to
-lt (small L)	<	less than
-le (small L)	<=	less than or equal to
-gt	>	greater than
-ge	>=	greater than or equal to

if statement

Syntax

```
if [ condition ]; then
```

```
# Commands to execute if the condition is true
```

```
fi
```

Example

```
#!/bin/bash
```

```
echo -n "Enter a number: "
```

```
read number
```

```
if [ $number -gt 10 ]; then
```

```
    echo "The number is greater than 10."
```

```
fi
```

if-else statement

Syntax

```
if [ condition ]; then
```

```
# Commands to execute if the condition is true
```

```
else
```

```
# Commands to execute if the condition is false
```

```
fi
```

Example

```
#!/bin/bash

echo -n "Enter a number: "

read number

if [ $number -gt 10 ]; then

    echo "The number is greater than 10."

else

    echo "The number is 10 or less."

fi
```

elif statement

```
if [ condition1 ]; then

    # Commands to execute if condition1 is true

elif [ condition2 ]; then

    # Commands to execute if condition is true

else

    # Commands to execute if none of the conditions are true

fi
```

Example

```
#!/bin/bash

echo -n "Enter a number: "

read number

if [ $number -gt 10 ]; then

    echo "The number is greater than 10."

elif [ $number -eq 10 ]; then

    echo "The number is exactly 10."

else

    echo "The number is less than 10."

Fi
```

for loop

Syntax

```
for item in list;

do

# Commands to execute

done
```

Example

```
#!/bin/bash

for i in 1 2 3 4 5; do    OR   for i in {1..5}; do    OR       for ((i = 1; i <= 5; i++)); do

echo "Number: $i"

done
```


while loop

```
while [ condition ]; do  
    # Commands to execute  
done
```

Example

```
#!/bin/bash  
  
count=1  
  
while [ $count -le 5 ]; do  
    echo "Count: $count"  
    ((count++)) // way to increment count by + 1  
done
```